트위캐스 자동 녹화 프로그램

.

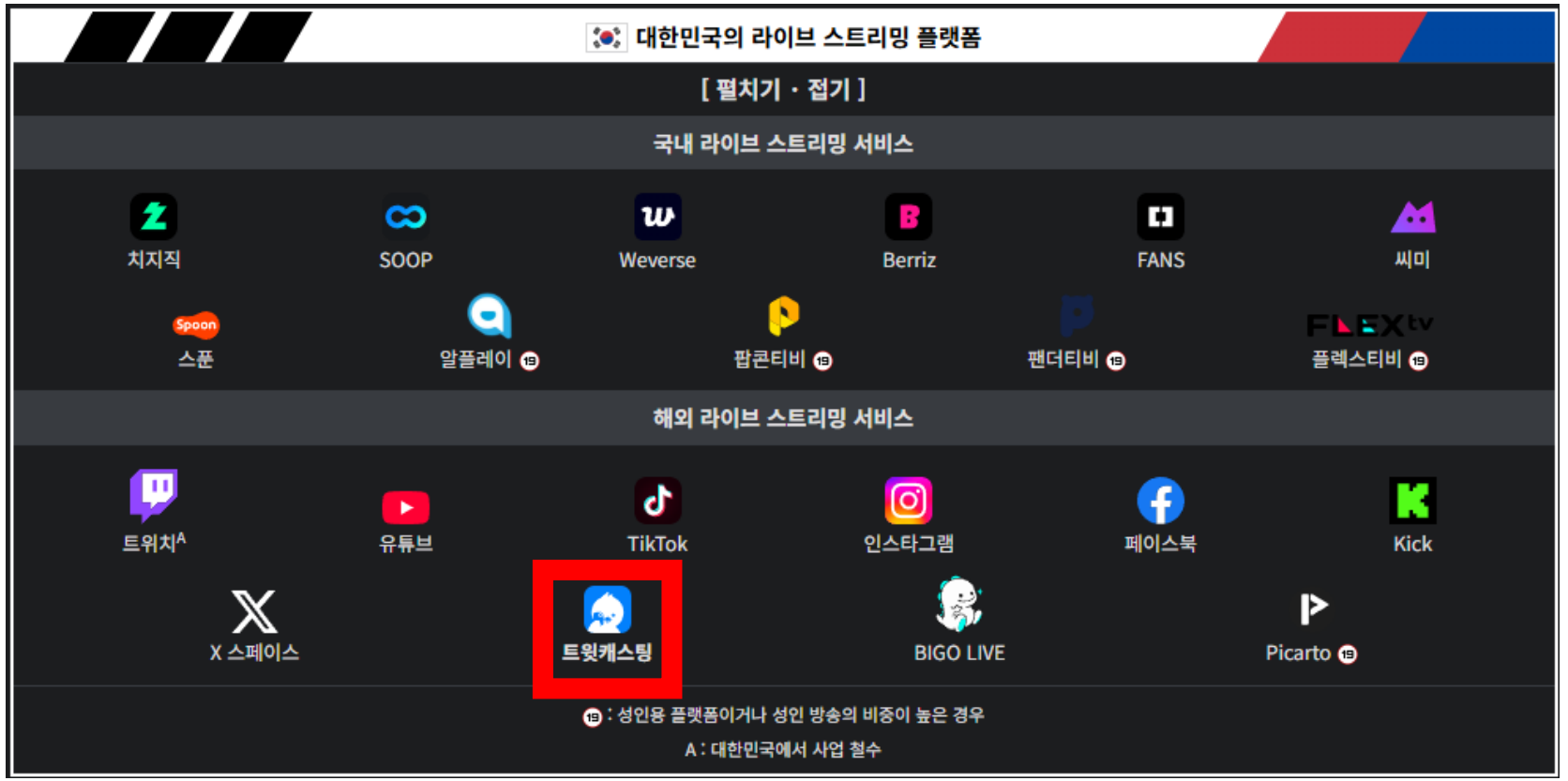
김학재

2026.10.15

목차

1. 라이브스트리밍 플랫폼
2. 영상녹화방식
3. WebSocket vs Polling
4. HLS

라이브 스트리밍 플랫폼

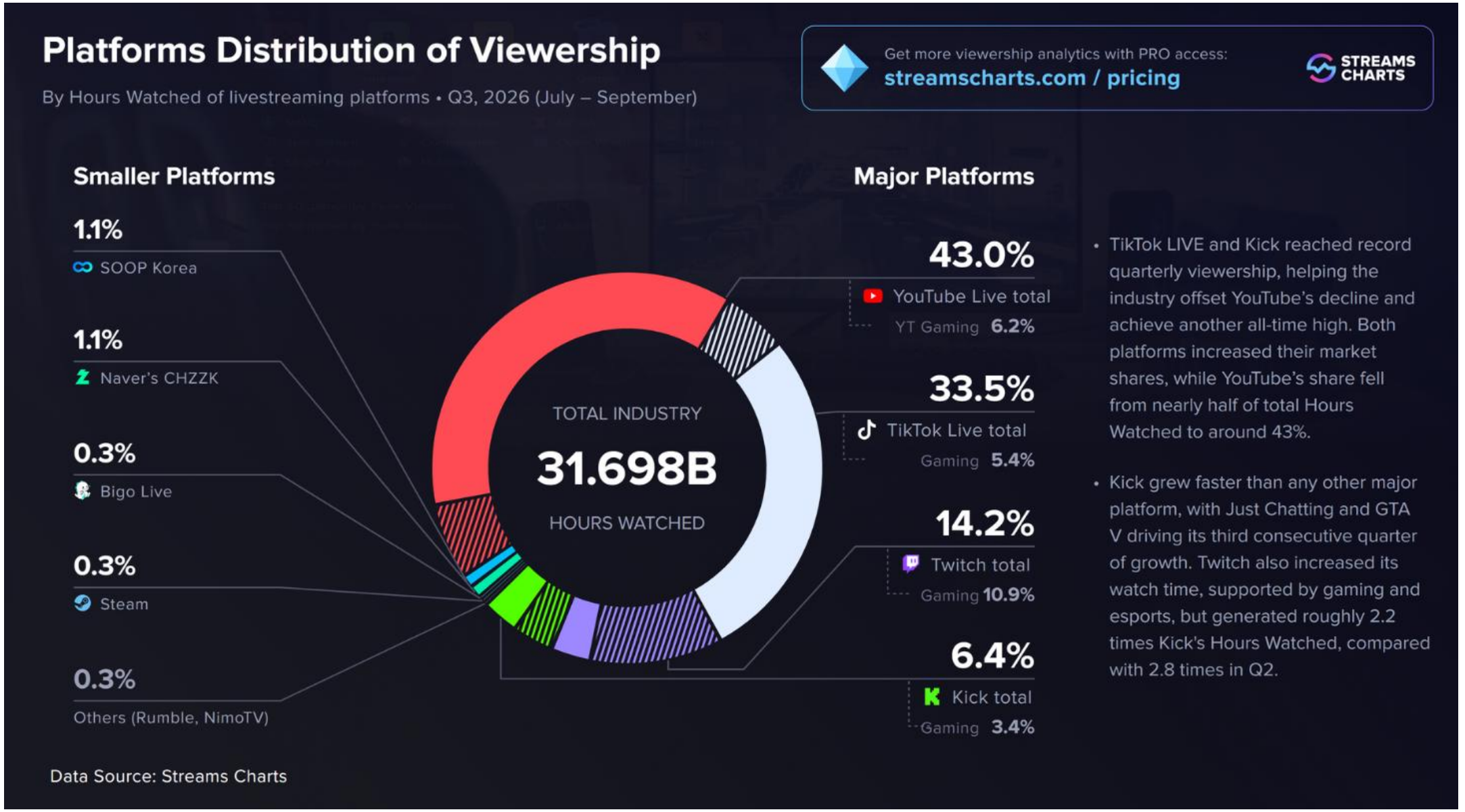




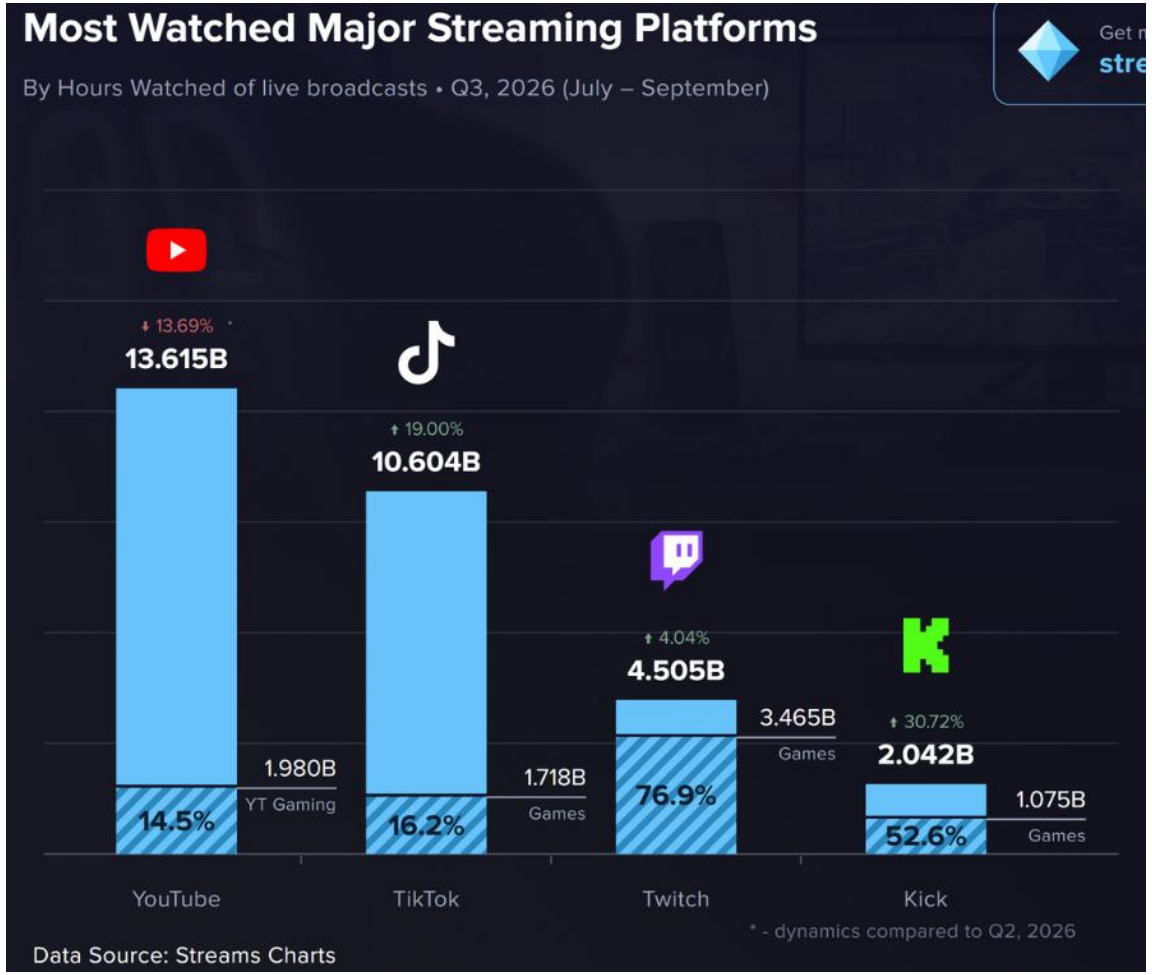
TwitCasting = Twitter + Broadcasting

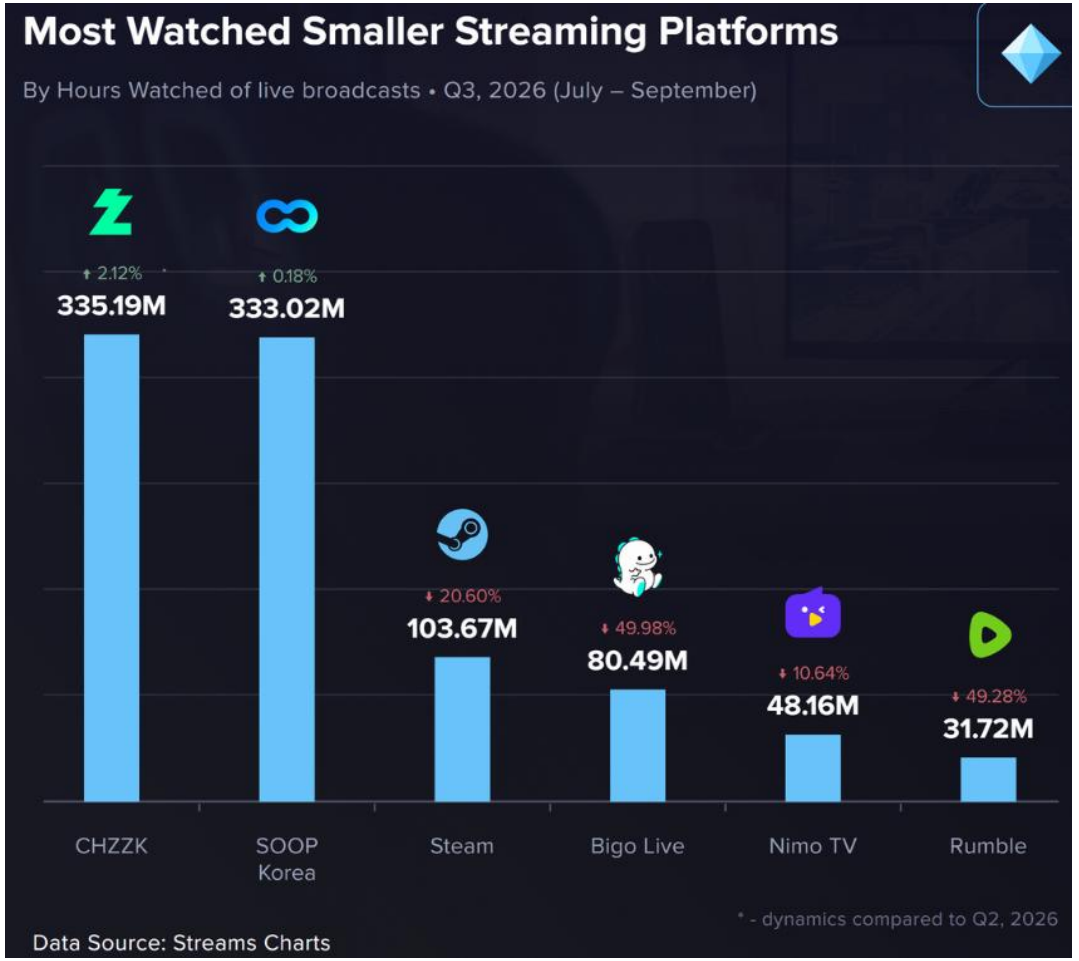
트위캐스는 트위터 연동 라이브 앱으로 시작했지만,
지금은 트위터보다 모바일 라이브 플랫폼에 가깝다.

26년 3분기 - 시청시간

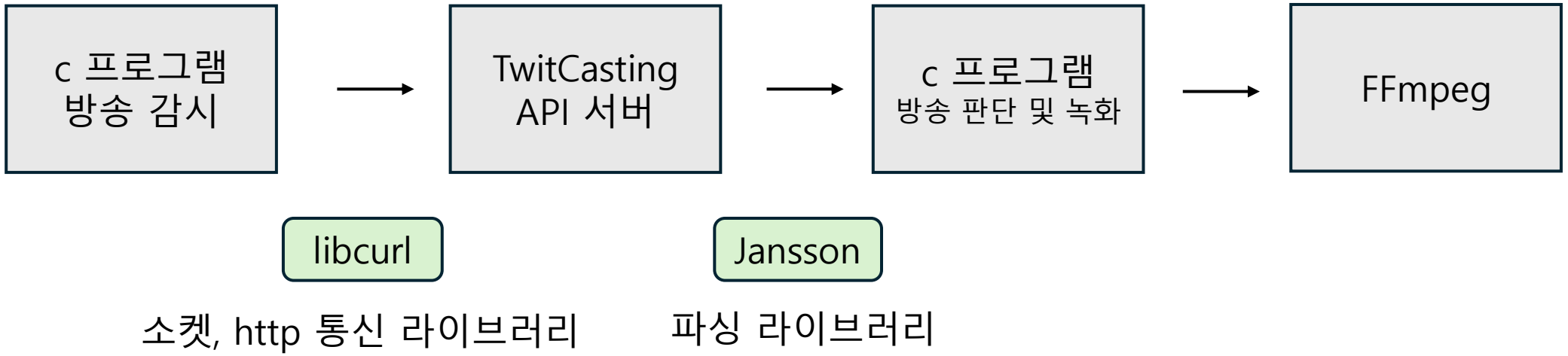

Platforms Distribution of Viewership
By Hours Watched of livestreaming platforms • Q3, 2026 (July – September)
Get more viewership analytics with PRO access:
streamscharts.com / pricing
STREAMS CHARTS
Smaller Platforms
1.1%
SOOP Korea
1.1%
Naver's CHZZK
0.3%
Bigo Live
0.3%
Steam
0.3%
Others (Rumble, NimoTV)
TOTAL INDUSTRY
31.698B
HOURS WATCHED
Major Platforms
43.0%
YouTube Live total
YT Gaming 6.2%
33.5%
TikTok Live total
Gaming 5.4%
14.2%
Twitch total
Gaming 10.9%
6.4%
Kick total
Gaming 3.4%
• TikTok LIVE and Kick reached record quarterly viewership, helping the industry offset YouTube's decline and achieve another all-time high. Both platforms increased their market shares, while YouTube's share fell from nearly half of total Hours Watched to around 43%.
• Kick grew faster than any other major platform, with Just Chatting and GTA V driving its third consecutive quarter of growth. Twitch also increased its watch time, supported by gaming and esports, but generated roughly 2.2 times Kick's Hours Watched, compared with 2.8 times in Q2.
Data Source: Streams Charts

시청시간


Most Watched Major Streaming Platforms
By Hours Watched of live broadcasts • Q3, 2026 (July – September)
13.69%
13.615B
1.980B
YT Gaming
14.5%
YouTube
19.00%
10.604B
1.718B
Games
16.2%
TikTok
4.04%
4.505B
3.465B
Games
76.9%
Twitch
30.72%
2.042B
1.075B
Games
52.6%
Kick
* - dynamics compared to Q2, 2026
Data Source: Streams Charts


Most Watched Smaller Streaming Platforms
By Hours Watched of live broadcasts • Q3, 2026 (July – September)
2.12%
335.19M
CHZZK
0.18%
333.02M
SOOP Korea
20.60%
103.67M
Steam
49.98%
80.49M
Bigo Live
10.64%
48.16M
Nimo TV
49.28%
31.72M
Rumble
* - dynamics compared to Q2, 2026
Data Source: Streams Charts

영상녹화 방식


c 프로그램
방송 감시
TwitCasting
API 서버
c 프로그램
방송 판단 및 녹화
FFmpeg
libcurl
Jansson
소켓, http 통신 라이브러리
파싱 라이브러리

libcurl 및 HTTP

인터넷 통신을 할 수 있도록 도와주는

HTTP, HTTPS 등 여러 프로토콜 지원하는 통신 라이브러리

libcurl : library client url

HTTP : Hypertext Transfer Protocol

웹에서 브라우저와 서버가 데이터를 주고받기 위한 프로토콜

HTTPS : Hypertext Transfer Protocol Secure

SSL : Secure Socket Layer
TLS : Transport Layer Security

상태코드

200 OK
401 Unauthorized
403 Forbidden
404 Not Found
500 Internal Server Error

코덱 및 컨테이너

코덱 codec : 영상·음성을 압축하거나 압축 해제하는 방식.
H.264 : 호환성이 좋고 압축 효율이 높은 영상 코덱
H.265(HEVC) : H.264보다 압축 효율이 높은 영상 코덱
AAC : 음질 대비 용량 효율이 좋은 음성 코덱

컨테이너 container : 영상·음성 스트림 등을 한 파일 안에 담는 형식.
MP4 : 호환성이 좋고 범용적으로 사용되는 미디어 컨테이너
MPEG_TS : 전송 중 오류에 비교적 강해 실시간 방송 및 HLS 스트리밍에 자주 사용되는 컨테이너
MKV : 여러 영상 음성 트랙과 자막 등을 유연하게 담을 수 있는 컨테이너

fork() , exec()

fork() : 현재 프로세스를 복제해 새로운 자식프로세스 생성

exec() : 해당 프로세스에서 실행하는 프로그램을 바꾼다.

여기서는 FFmpeg 실행하게 바꿈.

이렇게 하면

부모 프로세스는 방송을 계속 감시.

자식 프로세스는 ffmpeg로 녹화 진행.

FFmpeg로 HLS 수신
영상음성 데이터를 바로 복사
MPEG-TS형식으로 저장

```
pid_t pid = fork();

if (pid < 0) {
    perror("fork 실패");
    return -1;
}

if (pid == 0) {
    signal(SIGINT, SIG_DFL);
    signal(SIGTERM, SIG_DFL);
    signal(SIGCHLD, SIG_DFL);

    execlp(
        "ffmpeg",
        "ffmpeg",
        "-hide_banner",
        "-loglevel", "info",
        "-n",
        "-user_agent",
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 Chrome/146.0.0.0 Safari/537.36",
        "-headers",
        "Referer: https://twitcasting.tv/\r\n"
        "Origin: https://twitcasting.tv\r\n",
        "-live_start_index", "-1",
        "-i", hls_url,
        "-c", "copy",
        "-f", "mpegts",
        filename,
        (char *)NULL
    );

    perror("ffmpeg 실행 실패");
    _exit(127);
}
```

녹화 종료 절차

```
235  static void stop_recorder(pid_t *pid)
236  {
237      if (*pid <= 0)
238          return;
239
240      pid_t child = *pid;
241
242      log_event("녹화 종료 요청 중...\n");
243
244      if (kill(child, SIGINT) < 0 && errno != ESRCH)
245          perror("FFmpeg SIGINT 전송 실패");
246
247      int result = wait_for_child(child, STOP_TIMEOUT_SECONDS);
248
249      if (result == 0) {
250          log_event("FFmpeg가 종료되지 않아 SIGTERM을 보냅니다.\n");
251
252          if (kill(child, SIGTERM) < 0 && errno != ESRCH)
253              perror("FFmpeg SIGTERM 전송 실패");
254
255          result = wait_for_child(child, 2);
256      }
```

```
258      if (result == 0) {
259          log_event("FFmpeg 강제 종료를 시도합니다.\n");
260
261          if (kill(child, SIGKILL) < 0 && errno != ESRCH)
262              perror("FFmpeg SIGKILL 전송 실패");
263
264          int status;
265          while (waitpid(child, &status, 0) < 0) {
266              if (errno == EINTR)
267                  continue;
268
269              if (errno != ECHILD)
270                  perror("FFmpeg 최종 대기 실패");
271
272              break;
273          }
274      }
275
276      *pid = -1;
277      log_event("녹화 프로세스 종료 처리 완료\n");
278  }
```

SIGINT : Ctrl + C 인터럽트 시그널
SIGTERM : 정상 종료 요청
SIGKILL : 강제 종료

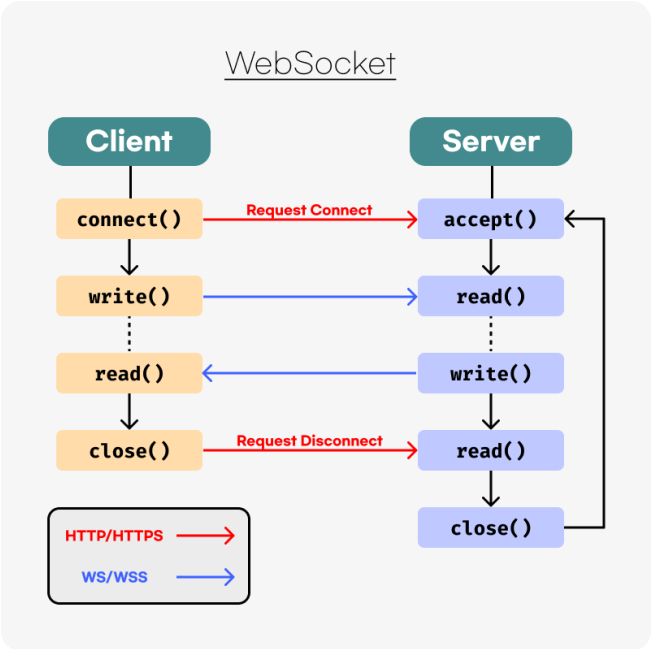
WebSocket vs Polling

WebSocket :

서버가 이벤트 발생시 이벤트 보내줌

단일 TCP연결로 동시양방향통신 채널을 제공하는 컴퓨터 통신 프로토콜

전이중 : 양방향으로 동시에 데이터 송수신가능



방송 시작/종료 이벤트를 실시간으로 받기

ws / wss (웹소켓 프로토콜) 사용

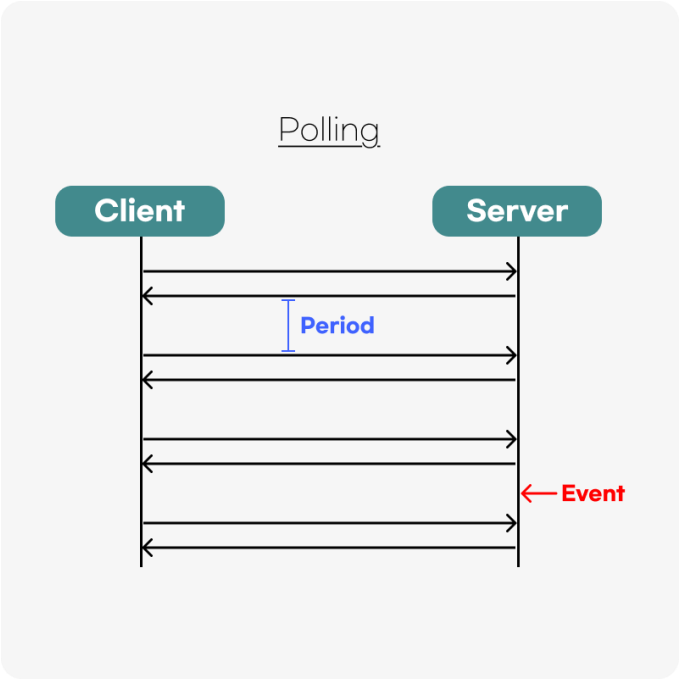
연결이 계속 지속됨

Polling :

클라이언트가 서버한테 주기적으로 물어보는 방식

반이중 : 양방향 통신, 단, 한번에 한쪽만 송신

x초마다 요청하고 응답받기를 반복.



HTTPS 프로토콜 사용

연결 계속 다시 해줘야함

polling 방식

keep running = 1

INTERVAL_SECONDS = 5

방송 상태 감지를 5초씩 하도록 하였다.

sleep(5)가 아닌 for문에 sleep(1)로 5번 돌리는 이유

중간에 Ctrl + C 같은 종료 요청에 좀 더 빠르게 반응하도록

```
while (keep_running) {

        ...방송 상태 확인...

    for (int i = 0;
         i < INTERVAL_SECONDS && keep_running;
         i++) {
        sleep(1);
    }
}
```

HLS

HLS : HTTP Live Streaming

유튜브나 인터넷 방송을 볼 때 영상을 처음부터 끝까지 다운로드할 때까지 기다리지 않음.
HLS는 영상을 작은 조각으로 나누고 필요한 조각을 순서대로 받아서 재생하는 방식

애플이 개발함

*HLS is designed for reliability and **dynamically adapts to network conditions** by optimizing playback for the available speed of wired and wireless connections.*

HLS는 안정성을 위해 설계되었으며 유무선 연결의 사용 가능한 속도에 맞게 재생을 최적화하여

네트워크 조건에 동적으로 적응합니다.

HLS

동영상 조각내기

Encoding(변환) : 원본 영상을 여러가지 화질 (1080p, 720p, 480p 등)로 먼저 변환

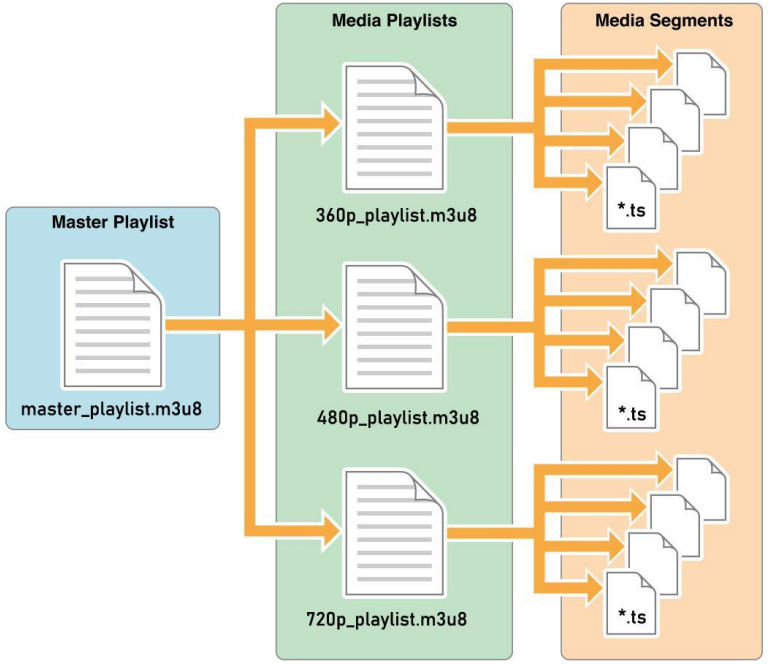
Segmentation(조각내기) : 변환한 영상을 6~10초 짜리 조각파일로 쪼갠다. (.ts or .m4s)

m3u8 : 쪼개놓은 영상들의 순서 목록 파일

```
#EXTINF:10.0,
video_001.ts
#EXTINF:10.0,
video_002.ts
#EXTINF:10.0,
video_003.ts
```

인터넷 속도에 따라 선택

1080p_video.m3u8
480p_video.m3u8



ABR 알고리즘

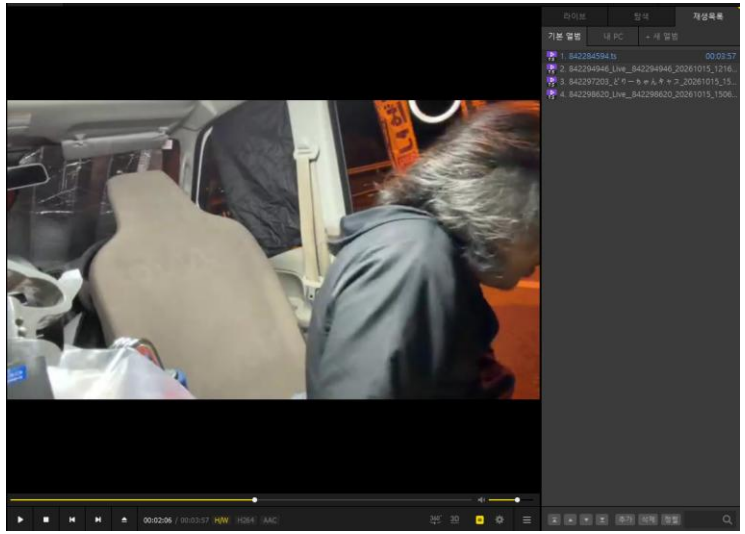
ABR : Adaptive Bitrate Streaming
네트워크 상황에 맞춰서 영상 화질 바꾸는 기능

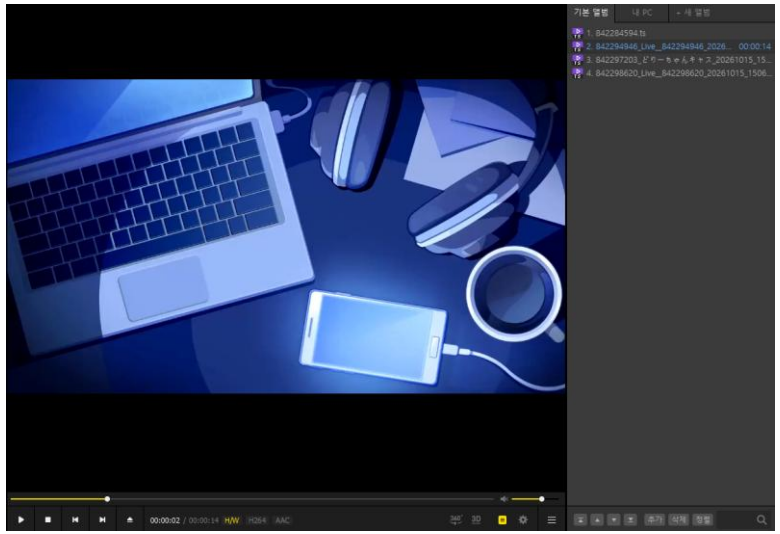
HLS
Master Playlist
- 1080p, 720p, 480p

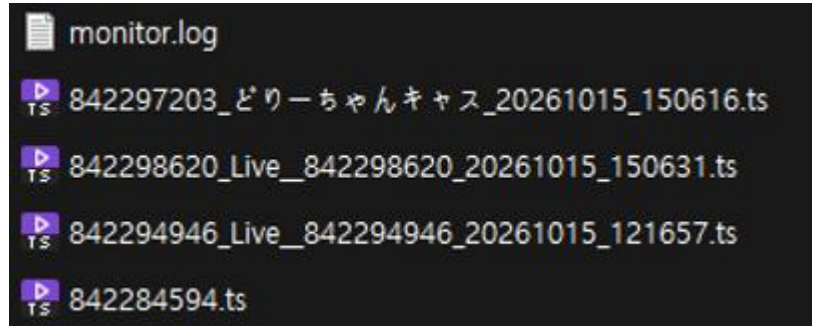
어떤 마스터 플레이리스트를 선택하냐
여러가지 알고리즘 존재.

1. 대역폭 기반 – 최근 영상 조각 다운로드 하는 속도 측정
2. 버퍼 기반 – 앞으로 재생할 영상이 얼마나 쌓여 있는지 보고 선택
3. 하이브리드 – 다운로드 속도와, 버퍼 상태 함께 고려

결과








```
k@k:/mnt/c/Users/k/OS/AutoRecord$ ./monitor
다중 계정 감시 시작 (2개)
감시 대상: NinjinChan_0u0
감시 대상: c:hrk3
녹화 폴더: recordings/
로그 파일: recordings/monitor.log
종료하려면 Ctrl+C를 누르세요.
[NinjinChan_0u0] 방송 중 | ID: 842297203 | 제목: どりーちゃんキャス
[NinjinChan_0u0] 방송 감지. 5초 후 녹화를 시작합니다.
[c:hrk3] 현재 방송 정보를 찾을 수 없습니다.
[NinjinChan_0u0] 방송 중 | ID: 842297203 | 제목: どりーちゃんキャス
녹화 시작: recordings/842297203_どりーちゃんキャス_20261015_150616.ts
[NinjinChan_0u0] 녹화를 시작했습니다.
[c:hrk3] 현재 방송 정보를 찾을 수 없습니다.
```

개선할점

1. 24시간 안정적으로 동작하는 환경 구축
2. 좀 더 쉽게 감시할 계정 추가 제거
3. C++로 변환후 객체지향 설계
4. WebSocket 방식 재도전
5. 내 프로그램에서는 FFmpeg가 HLS의 어떤 마스터 플레이리스트를 가져오는지 더 정확히 분석

감사합니다